Devel Documentation

SaltOS 4.1 r2757

October 2026

Contents

1 Introduction **6**

1.1 Advanced System Features . . . 6

1.2 Architecture Overview . . . 6

1.2.1 Client Layer (SPA) . . . 6

1.2.2 API Layer (Execution Engine) . . . 6

1.2.3 Declarative Application Layer . . . 7

1.2.4 Data & Persistence Layer . . . 7

1.2.5 Cross-Cutting Concerns . . . 7

1.2.6 Design Principles . . . 8

1.3 Project Structure . . . 8

1.4 Code Directory . . . 8

1.5 Data Directory . . . 9

2 Quick Setup with Debian 13 **9**

2.1 Install Debian 13 (netinst) . . . 9

2.2 Install PHP and required packages . . . 9

2.3 Download and extract SaltOS4 . . . 10

2.4 Configure SQLite3 database access . . . 10

2.5 Run the initial setup . . . 10

2.6 Optional: use MariaDB or MySQL instead of SQLite3 . . . 10

3 Backend (api/) **11**

3.1 Autoloaded Modules (‘php/autoload/‘) . . . 11

3.2 Database Drivers (‘php/database/‘) . . . 12

3.3 Libraries (‘php/lib/‘) . . . 12

3.4 Action Modules (‘php/action/‘) . . . 13

3.5 Other API Components . . . 14

3.6 Configuration Files . . . 14

3.6.1 dbschema.xml . . . 14

3.6.2 dbstatic.xml . . . 14

3.6.3 config.xml . . . 14

3.6.4 cron.xml . . . 14

3.6.5 locale.xml . . . 15

3.6.6 bs_theme.xml . . . 15

3.6.7 css_theme.xml . . . 15

3.7 API Access . . . 15

3.7.1 Web server access (Apache, Nginx, Cherokee) . . . 15
3.7.2 Command-line access (CLI) . . . 15
3.8 API Authentication . . . 15
3.8.1 HTTP Access . . . 16
3.8.2 CLI Access . . . 16
3.9 Token Lifecycle & Session Security . . . 16
3.9.1 Token Binding . . . 17
3.9.2 Expiration and Invalidation . . . 17
3.9.3 CLI Considerations . . . 17
3.9.4 Additional Protections . . . 17

4 Frontend (web/) 17

4.1 Logic (core.js and app.js) . . . 18
4.1.1 core.js Overview . . . 18
4.1.2 app.js Overview . . . 18
4.1.3 Integration and Responsibilities . . . 19
4.2 Custom Widgets . . . 19
4.2.1 The element.saltos object . . . 19
4.2.2 Rules . . . 19
4.2.3 Example . . . 20
4.3 Deployment . . . 22
4.4 Offline Support (Service Worker) . . . 22
4.4.1 Main Features . . . 22
4.4.2 Internal Structure . . . 22
4.4.3 Debugging and Observability . . . 23
4.4.4 Summary . . . 23
4.5 Authentication . . . 23
4.5.1 Core Auth Module . . . 23
4.5.2 Login Workflow . . . 23
4.6 Build System . . . 24
4.6.1 Overview of Modes . . . 24
4.6.2 Build Process with Makefile . . . 24
4.6.3 Web Directory Overview . . . 25
4.6.4 Comparison Table . . . 25

5 Application System 25

5.1 Manifest Files . . . 25
5.1.1 General Structure . . . 26

5.1.2 Groups (<group>) . . . 26
5.1.3 Apps (<app>) . . . 26
5.1.4 Permission System . . . 27
5.1.5 Feature Modules . . . 28
5.1.6 Special Cases . . . 29
5.1.7 Example . . . 29
5.1.8 Usage Notes . . . 30
5.2 YAML-based Apps . . . 30
5.2.1 tokenslog.yaml . . . 30
5.2.2 configlog.yaml . . . 31
5.2.3 taxes.yaml . . . 32
5.2.4 Advanced YAML file documentation . . . 32
5.2.5 YAML structure . . . 33
5.2.6 Explanation of YAML fields . . . 33
5.2.7 List and Form schema . . . 34
5.2.8 Select section . . . 34
5.2.9 Attr section . . . 34
5.2.10 Summary . . . 34
5.3 XML-based Complex Apps . . . 34
5.3.1 invoices.xml . . . 34
5.4 Available Layouts . . . 35
5.4.1 Type1 based layouts . . . 36
5.4.2 Type2 based layouts . . . 36
5.4.3 Type3 layout . . . 36
5.4.4 Mobile adaptation . . . 36
5.5 Data Model using dbschema.xml . . . 36
5.5.1 Structure of dbschema.xml . . . 36
5.5.2 Example . . . 37
5.5.3 Automatic Schema Management . . . 37
5.5.4 System-Wide and App-Specific Schemas . . . 37
5.5.5 Benefits of dbschema.xml . . . 37
5.5.6 Complement: dbstatic.xml . . . 38
5.6 Static Data using dbstatic.xml . . . 38
5.7 How to Add a New App . . . 38
5.7.1 Create the App Folder . . . 38
5.7.2 Define the Manifest . . . 38
5.7.3 Define the Database Schema . . . 39

5.7.4 Define the Static Data (Optional) . . . 39
5.7.5 Add the Application Definition . . . 39
5.7.6 Ensure Symbolic Links . . . 40
5.7.7 Run Database Setup . . . 40
5.7.8 Assign Permissions . . . 40
5.7.9 Test the App . . . 40

6 Makefile Overview **40**

6.1 Build Targets . . . 40
6.2 Documentation . . . 41
6.3 Testing . . . 41
6.4 Environment Checks . . . 42
6.5 Dependency Management . . . 42
6.6 Code Statistics . . . 42
6.7 System Setup . . . 43
6.8 System Actions . . . 43
6.9 Local Web Server . . . 43
6.10 Docker . . . 44
6.11 System langs . . . 44
6.12 Script Directory Overview . . . 45
6.12.1 Build and Asset Generation . . . 45
6.12.2 Testing and Quality . . . 45
6.12.3 Database and Migration . . . 46
6.12.4 Dependency and Library Management . . . 46
6.12.5 Docker and Containerization . . . 46

1 Introduction

SaltOS4 is a modular, extensible framework for building Rich Internet Applications (RIAs). It is designed for rapid development of dynamic, offline-capable applications using a clean separation between backend and frontend.

It is composed of:

- A PHP backend (‘api/‘)
- A JavaScript frontend (‘web/‘)
- REST/JSON API with support for CLI execution
- Offline support through a Service Worker proxy
- Declarative and dynamic app definitions via XML and/or YAML
- Shared templates and logic to reduce boilerplate

1.1 Advanced System Features

SaltOS4 includes two key functionalities that stand out for their utility and focus on traceability:

- Version traceability of records: Each modification in application data generates a new version of the record, maintaining a complete change history, similar to version control systems like Subversion or Git.
- Data access logging: The system records who accessed a piece of data, from where, and in what context (e.g., if it was shown in a list or form). This provides full auditability of data usage and consultation.

1.2 Architecture Overview

SaltOS4 follows a layered and declarative architecture designed to separate responsibilities clearly between frontend, backend, configuration, and data persistence.

The system operates across four main layers:

1.2.1 Client Layer (SPA)

The frontend is a Single Page Application (SPA) written in JavaScript and located in ‘web/‘.

- ‘core.js‘ provides low-level utilities (AJAX, DOM helpers, error handling).
- ‘app.js‘ manages UI logic, navigation, layout, and interaction.
- Additional modules (auth, form, filter, etc.) extend functionality.
- A Service Worker (‘proxy.js‘) intercepts network requests to provide caching, offline support, and request queueing.

All user interactions eventually result in REST-style requests to the backend.

1.2.2 API Layer (Execution Engine)

The backend is implemented in PHP under ‘api/‘.

- ‘index.php‘ acts as the main entry point.
- Autoloaded modules provide core services (database, permissions, logging, tokens, etc.).
- Action modules (‘action/*.php‘) dispatch REST or CLI requests.
- The same execution path supports both HTTP and CLI access.

Requests follow this simplified lifecycle:

- Request received (HTTP or CLI)
- Token validation (if required)
- Permission validation
- Action dispatch
- Response serialization (JSON)

1.2.3 Declarative Application Layer

Applications are defined declaratively using:

- ‘manifest.yaml‘ → metadata and permissions
- ‘dbschema.xml‘ → database structure
- ‘dbstatic.xml‘ → static data
- ‘*.yaml‘ or ‘*.xml‘ → UI definitions

For YAML-based apps:

- YAML is transformed into XML dynamically.
- The generated XML is cached for performance.
- The execution engine processes XML definitions to render UI and perform actions.

This approach enables rapid development without writing boilerplate controllers or views.

1.2.4 Data & Persistence Layer

SaltOS4 supports:

- MariaDB/MySQL (with optional Mroonga full-text indexing)
- SQLite (lightweight deployments)

Schema management is automatic:

- ‘dbschema.xml‘ definitions are compared with the live database.
- Differences are applied automatically.
- No manual migration scripts are required.

Versioning, logging, indexing, and feature modules (‘has_*‘) extend the data layer transparently.

1.2.5 Cross-Cutting Concerns

Several features operate across all layers:

- Token-based authentication (HTTP and CLI)
- Ownership and permission control
- Version tracking with blockchain-style integrity
- Access logging
- Automatic schema synchronization
- Offline support via Service Worker

1.2.6 Design Principles

SaltOS4 is built around the following principles:

- Declarative over imperative
- Convention over configuration
- Automatic synchronization of schema and metadata
- Unified execution model (HTTP and CLI share the same engine)
- Modular, extensible app architecture
- Minimal boilerplate for CRUD applications

This architecture allows SaltOS4 to function both as a rapid application framework and as a structured business application platform.

1.3 Project Structure

The root directory of SaltOS4 is organized as follows:

- ‘README.md‘: Provides an overview and quick-start instructions for the project.
- ‘LICENSE.md‘: Text of the open-source license under which SaltOS4 is released.
- ‘CONTRIBUTING.md‘: Setup steps, Docker profiles, coding standards and project structure for contributors.
- ‘CHANGELOG.md‘: Notable changes of each release, with their upgrade notes.
- ‘SECURITY.md‘: Security policy, supported versions and how to report a vulnerability.
- ‘AGENTS.md‘: Instructions and known pitfalls for the AI agents that work with the code.
- ‘makefile‘: Automates repetitive development tasks, such as generating documentation using the scripts in ‘scripts/‘.
- ‘code/‘: Contains the full source code of the system, split into backend (PHP) and frontend (JavaScript) modules.
- ‘docs/‘: Documentation files written in ‘.t2t‘ format, including generated outputs (‘.pdf‘, ‘.html‘) and the source files used to build them.
- ‘scripts/‘: Collection of PHP utilities that extract documentation from code comments, produce ‘.t2t‘, ‘.pdf‘, or ‘.html‘ outputs and more...
- ‘ujest/‘: JavaScript unit tests using Jest.
- ‘utest/‘: PHP unit tests using PHPUnit.

This structure keeps the system cleanly separated between source code, documentation, automation, and testing components.

1.4 Code Directory

The ‘code/‘ directory contains the source code and runtime data of SaltOS4, structured as follows:

- ‘api/‘: Backend core written in PHP. Handles REST endpoints, internal services, and logic.
- ‘apps/‘: Application modules. Each app includes its own backend, frontend, and configuration (PHP, JS, XML), plus an optional ‘lib/‘ for its own vendored third-party dependencies (e.g. ‘apps/emails/lib/mailmimeparser‘, ‘apps/certs/lib/fpdi‘) - parallel to ‘api/lib/‘ but scoped to that one app.
- ‘data/‘: Stores persistent and temporary data generated by the system, such as files, images, downloaded or sent emails, cache, and temp files.
- ‘web/‘: Frontend resources - primarily JavaScript and CSS - used in the browser interface.

This structure supports both application logic and data flow across the SaltOS4 system.

1.5 Data Directory

SaltOS4 stores runtime and persistent data under the ‘data/‘ directory. This folder contains several subdirectories used for caching, file storage, logging, and background task management. It is essential for the proper operation of the system.

- ‘cache/‘: Stores cached data to improve performance.
- ‘cron/‘: Contains execution records and temporary files for scheduled tasks.
- ‘files/‘: Persistent file storage used by applications.
- ‘inbox/‘: Incoming data such as fetched emails.
- ‘logs/‘: Application and system logs.
- ‘outbox/‘: Outgoing data, such as email messages.
- ‘temp/‘: Temporary files used during various operations.
- ‘trash/‘: Items marked as deleted but not permanently removed.
- ‘upload/‘: Staging area for newly uploaded files before they are processed.

Make sure this directory is writable by the web server and CLI user.

2 Quick Setup with Debian 13

This section explains how to install SaltOS4 from scratch using a minimal Debian 13 (netinst) environment and SQLite3 as the backend. It also includes optional steps to configure MariaDB and enable Mroonga for full-text search support.

2.1 Install Debian 13 (netinst)

During the installation, do **not** select any graphical environment. Only select the **SSH server** and **Web server** task options.

After the system is installed and you log in, install some minimal utilities:

```
sudo apt install net-tools sudo vim
```

This ensures you have access to ‘ifconfig‘, ‘sudo‘ (configure it for your user), and ‘vim‘.

2.2 Install PHP and required packages

Install PHP with the required dependencies and the web server that you prefer, nginx with PHP-FPM:

```
sudo apt install php-fpm php-mbstring php-xml php-gd php-mysql php-sqlite3 php-curl php-
    zip nginx
sudo systemctl restart php*-fpm nginx
```

Or apache with the PHP module:

```
sudo apt install php php-mbstring php-xml php-gd php-mysql php-sqlite3 php-curl php-zip
sudo systemctl restart apache2
```

In both cases, configure the web server with the recipe of ‘scripts/‘ that matches it (‘server.nginx.conf‘ or ‘server.apache.conf‘).

To enable the digital signature feature in SaltOS4, install the following packages:

```
sudo apt install poppler-utils libnss3-tools
```

Note: this packages provide commands like pdfsig and certutil.

2.3 Download and extract SaltOS4

Download the SaltOS4 code outside of the default document root, only the ‘web/‘ directory must be published:

```
cd /var/www
sudo wget -L --content-disposition https://download.saltos.org/
sudo tar xvzf SaltOS4-master.tar.gz
cd SaltOS4-master/code
chmod ugo+rwx data/*
```

Then use ‘/var/www/SaltOS4-master/code/web‘ as the path of the web directory in the recipe of your web server.

2.4 Configure SQLite3 database access

If you are using the Makefile (recommended), no manual configuration is required. The target ‘make setupsqlite‘ automatically creates the correct database configuration file.

If you prefer manual configuration instead of using Make, create the following file:

```
echo '<root><db><type>pdo_sqlite</type></db></root>' > data/files/config.xml
```

2.5 Run the initial setup

If you are using the Makefile (recommended), make sure the project root contains the Makefile and required symbolic links. You can verify the environment with:

```
make check
```

Then initialize the system with SQLite:

```
make setupsqlite
make setupdemo
```

If you prefer MariaDB/MySQL:

```
make setupmysql
make setupdemo
```

Use ‘make setuponly‘ instead of ‘make setupdemo‘ to install only the core schema and static data, without demo data.

‘make setupall‘ is a full reset, not an alternative install: it drops the ‘saltos‘ MariaDB database, empties ‘data/*‘, installs demo data on both MySQL and SQLite, and finally removes ‘config.xml‘ (back to the default engine, MySQL). It ignores any engine selected before, and it destroys existing data.

If you prefer manual CLI execution instead of Make:

```
php api/index.php setup
```

To install demo data manually:

```
user=admin php api/index.php setup/certs
user=admin php api/index.php setup/company
user=admin php api/index.php setup/emails
user=admin php api/index.php setup/crm
user=admin php api/index.php setup/hr
user=admin php api/index.php setup/purchases
user=admin php api/index.php setup/sales
```

2.6 Optional: use MariaDB or MySQL instead of SQLite3

If you prefer MariaDB/MySQL, the easiest way to get a ready-to-use instance with Mroonga (full-text search) support, without compiling anything by hand, is the ‘db‘ Docker profile. It builds a MariaDB + Mroonga image from ‘scripts/Dockerfile.db‘

and runs it like a native system service - host networking, bind-mounted onto the real ‘/var/lib/mysql‘ and ‘/run/mysqld‘, so it behaves the same as an ‘apt‘-installed MariaDB reachable at ‘localhost:3306‘:

```
make dbbuild
make dbstart
```

On first boot this seeds ‘/var/lib/mysql‘, installs the Mroonga plugin, and creates the ‘saltos‘ database plus a ‘saltos‘ user (password ‘saltos‘). Manage it with ‘make dbstatus‘, ‘make dblogs‘, ‘make dbbash‘, and ‘make dbstop‘.

If you prefer installing MariaDB directly on the host instead:

```
sudo apt install mariadb-server
sudo systemctl restart mariadb
sudo mariadb -uroot <<EOF
CREATE DATABASE saltos;
CREATE USER 'saltos'@'localhost' IDENTIFIED BY 'saltos';
GRANT ALL PRIVILEGES ON saltos.* TO 'saltos'@'localhost';
FLUSH PRIVILEGES;
EOF
```

Optionally, you may enable the ‘mroonga‘ engine to get full-text support - check whether your distribution ships ‘mariadb-plugin-mroonga‘. If it doesn’t, ‘scripts/Dockerfile.db‘ is a ready-made recipe for building Mroonga from source against a matching MariaDB version, so you don’t have to work out the build steps yourself.

Then configure the database like this:

```
echo '<root><db><type>pdo_mysql</type><pdo_mysql host="localhost" port="3306" name="saltos
    " user="saltos" pass="saltos"/></db></root>' > data/files/config.xml
```

Note: replace ‘host‘, ‘port‘, ‘name‘, ‘user‘ and ‘pass‘ with your actual database settings.

3 Backend (api/)

The backend is under the ‘api/‘ folder and contains four folders (autoload, database, lib and action) to organize the code depending their usage and functionality.

3.1 Autoloaded Modules (‘php/autoload/‘)

Core functions automatically loaded on each request:

- ‘autoload/apps.php‘: * Apps helper module
- ‘autoload/array.php‘: * Array helper module
- ‘autoload/config.php‘: * Config helper module
- ‘autoload/database.php‘: * Database helper module
- ‘autoload/datetime.php‘: * Datetime helper module
- ‘autoload/error.php‘: * Error helper module
- ‘autoload/exec.php‘: * Execution helper module
- ‘autoload/file.php‘: * File utils helper module
- ‘autoload/getdata.php‘: * Get data helper module
- ‘autoload/gettext.php‘: * Gettext helper module
- ‘autoload/iniset.php‘: * Iniset helper module
- ‘autoload/json.php‘: * Json helper module
- ‘autoload/log.php‘: * Log helper module
- ‘autoload/memory.php‘: * Memory helper module

- ‘autoload/mime.php‘: * Mime helper module
- ‘autoload/output.php‘: * Output helper module
- ‘autoload/pcov.php‘: * PCOV helper module
- ‘autoload/perms.php‘: * Permissions helper module
- ‘autoload/semaphores.php‘: * Semaphore helper module
- ‘autoload/server.php‘: * Server helper module
- ‘autoload/sql.php‘: * SQL utils helper module
- ‘autoload/strings.php‘: * String utils helper module
- ‘autoload/system.php‘: * System helper module
- ‘autoload/tokens.php‘: * Tokens helper module
- ‘autoload/user.php‘: * User helper module
- ‘autoload/version.php‘: * Version helper module
- ‘autoload/xml2array.php‘: * XML to Array helper module
- ‘autoload/yaml.php‘: * Yaml helper module
- ‘autoload/zindex.php‘: * Main execution module

3.2 Database Drivers (‘php/database/‘)

Supports:

- ‘database/libsqlite.php‘: * SQLite3 functions library
- ‘database/mysqli.php‘: * MySQL improved driver
- ‘database/pdo_mssql.php‘: * PDO MsSQL driver
- ‘database/pdo_mysql.php‘: * PDO MySQL driver
- ‘database/pdo_pgsql.php‘: * PDO PostgreSQL driver
- ‘database/pdo_sqlite.php‘: * PDO SQLite driver
- ‘database/sqlite3.php‘: * SQLite3 driver

3.3 Libraries (‘php/lib/‘)

Not autoloaded; provide extra functionality:

- ‘lib/actions.php‘: * Actions module
- ‘lib/array2xml.php‘: * Array to XML helper module
- ‘lib/ascii.php‘: * Make Table ASCII
- ‘lib/auth.php‘: * Login functions
- ‘lib/barcode.php‘: * Barcode helper module
- ‘lib/captcha.php‘: * Captcha helper module
- ‘lib/color.php‘: * Color helper module
- ‘lib/control.php‘: * Control helper module
- ‘lib/cron.php‘: * Cron utils helper module
- ‘lib/dbschema.php‘: * Database schema helper module
- ‘lib/dbstats.php‘: * DB Stats module

- ‘lib/depend.php‘: * Dependencies feature
- ‘lib/export.php‘: * Export helper module
- ‘lib/files.php‘: * Files module
- ‘lib/gc.php‘: * Garbage collector helper module
- ‘lib/gdlib.php‘: * GD utils helper module
- ‘lib/import.php‘: * Import file helper module
- ‘lib/indexing.php‘: * Make index helper module
- ‘lib/license.php‘: * License and legal information helpers for SaltOS framework
- ‘lib/log.php‘: * Log helper module
- ‘lib/math.php‘: * Math utils helper module
- ‘lib/notes.php‘: * Notes module
- ‘lib/password.php‘: * Password helper module
- ‘lib/pdf.php‘: * PDF helper module
- ‘lib/push.php‘: * Push utils helper module
- ‘lib/qrcode.php‘: * QRCode helper module
- ‘lib/score.php‘: * Score image helper module
- ‘lib/server.php‘: * Server helper module
- ‘lib/setup.php‘: * Setup helper module
- ‘lib/trash.php‘: * Send file to trash
- ‘lib/unoconv.php‘: * Unoconv library
- ‘lib/upload.php‘: * Add upload file
- ‘lib/version.php‘: * Version helper module

3.4 Action Modules (‘php/action/‘)

Handle concrete system actions (CLI-aware where needed):

- ‘action/about.php‘: Exposes framework metadata, license information, third-party libraries and legal text as JSON.
- ‘action/add.php‘: Handles log or auxiliary add operations triggered by the system.
- ‘action/app.php‘: Main application dispatcher for ‘app/...‘ REST and CLI calls.
- ‘action/auth.php‘: Implements authentication endpoints (‘login‘, ‘check‘, ‘logout‘, ‘update‘).
- ‘action/cron.php‘: Executes scheduled tasks defined in ‘cron.xml‘.
- ‘action/gc.php‘: Executes garbage collection routines (cleanup of expired or temporary data).
- ‘action/image.php‘: Generates dynamic images (e.g., barcodes or other image-based outputs).
- ‘action/indexing.php‘: Triggers full-text indexing and index maintenance processes.
- ‘action/integrity.php‘: Executes integrity validation checks for records and system data.
- ‘action/push.php‘: Handles push-related operations and notification dispatching.
- ‘action/setup.php‘: Performs schema synchronization and static data initialization.
- ‘action/upload.php‘: Handles file upload processing and attachment management.

3.5 Other API Components

This section lists complementary elements in the backend that support the main API logic.

- ‘index.php‘: Main API entry point. It loads required bootstrap files (‘autoload‘, ‘zindex.php‘) and dispatches incoming REST or CLI requests.
- ‘img/‘: Contains SaltOS4 logos and branding assets.
- ‘locale/‘: Stores per-language ‘messages.yaml‘ translation files (‘en_US‘, ‘es_ES‘, ‘ca_ES‘).
- ‘xml/‘: Holds static configuration files (e.g. schema, themes, locales).
- ‘lib/‘: Contains third-party or integrated libraries used across the backend logic. Includes:
  - ‘atkinson‘, ‘gorrisans‘: font support
  - ‘edifact‘: EDI message parsing
  - ‘html2text‘: HTML-to-plain-text conversion (used for email and document processing)
  - ‘phpass‘: legacy password hashing for compatibility with older PHP installs
  - ‘phpspreadsheet‘: spreadsheet creation and parsing
  - ‘semver‘: version constraint checking (used by ‘check_composer()‘ to validate vendored libraries against the running PHP)
  - ‘tc-lib-pdf‘: used to import the Atkinson Hyperlegible font into the PDFs generated with TCPDF
  - ‘tcpdf‘, ‘yaml‘: PDF generation and YAML parsing
  - ‘zxcvbn‘: password strength estimation

3.6 Configuration Files

SaltOS4 relies on several XML configuration files to define system-wide behavior, themes, languages, scheduled tasks, and database initialization. These files are stored in the root configuration directories and are essential for proper system setup.

3.6.1 dbschema.xml

Defines the database schema structure used across all applications. It lists all tables, their fields, types, primary keys, indexes, and relationships.

3.6.2 dbstatic.xml

Populates static content in the database. This includes predefined record values required for the system to operate correctly (e.g., permission types, core options).

3.6.3 config.xml

Contains global system configuration, such as paths, timezones, defaults, UI settings, and other bootstrap parameters.

3.6.4 cron.xml

Specifies scheduled tasks and their timing. Each entry defines a script to be executed periodically, along with interval definitions and optional conditions.

3.6.5 locale.xml

Defines available languages and translation mappings for the interface. Used by the frontend and backend to provide multi-language support.

3.6.6 bs_theme.xml

Defines Bootstrap theme settings, including color schemes, spacing, and variables that adapt the interface visually.

3.6.7 css_theme.xml

Defines custom CSS style variants used in the frontend. Enables additional visual personalization beyond Bootstrap presets.

3.7 API Access

SaltOS4 supports access via HTTP or CLI.

3.7.1 Web server access (Apache, Nginx, Cherokee)

The main idea of sending information to SaltOS is to use the latest technologies, to do it, we are using restful request

Note: '@rest/x' and '@json/x' below are shorthand for the corresponding 'get_data('rest/x')' / 'get_data('json/x')' call - the function that reads these values in XML 'eval="true"' attributes and PHP action code, e.g. 'get_data('rest/1')' in 'code/apps/sales/xml/invoices.xml'.

- GET with REST:
  - An example request: 'https://host/api/?/app/invoices/view/2'
  - '@rest/0' = app
  - '@rest/1' = invoices
  - '@rest/2' = view
  - '@rest/3' = 2
- POST with JSON:
  - An example request: 'https://host/api/?/app/invoices/insert'
  - Use 'Content-Type: application/json'
  - Body parsed into '@json/...'

3.7.2 Command-line access (CLI)

- 'php api/index.php app/customers/view/100'
- 'user=admin php api/index.php app/customers/view/100'
- 'cat data.json | user=admin php api/index.php app/customers/insert'

3.8 API Authentication

SaltOS4 implements token-based authentication for both HTTP and CLI access. The authentication logic is handled by 'auth.php', which delegates to utility functions in 'php/lib/auth.php'. Tokens are validated based on IP address and user agent.

Supported authentication actions:

- ‘auth/login‘: Authenticates a user using ‘user‘ and ‘pass‘ (JSON fields). Returns the token with its creation and expiration dates.
- ‘auth/check‘: Validates the current token.
- ‘auth/logout‘: Invalidates the current token and ends the session.
- ‘auth/update‘: Changes the password of the currently authenticated user.

Each request is handled in ‘api/php/action/auth.php‘, which delegates to utility functions defined in ‘php/lib/auth.php‘.

The token is associated with the client IP address and user-agent to prevent misuse. Internally, ‘current_token()‘ retrieves the token from the request and validates it against stored sessions.

3.8.1 HTTP Access

Users authenticate by calling the ‘auth/login‘ endpoint with a JSON payload and receive a token in response. This token must be included in subsequent requests using the ‘Authorization‘ header.

Example of authenticating via ‘curl‘:

```
curl -X POST "https://yourdomain/api/?/auth/login" \
    -H "Content-Type: application/json" \
    -d '{"user":"admin", "pass":"secret"}'
```

Once authenticated, subsequent requests must include the token using a header:

```
curl "https://yourdomain/api/?/app/yourapp/list" \
    -H "Authorization: Bearer your-token-goes-here"
```

The backend automatically links the token to the user and group using functions from ‘autoload/user.php‘. Permissions are enforced using ‘autoload/perms.php‘, which determines whether a user can perform a given action on a given application or record.

3.8.2 CLI Access

Authentication also works from the command line using the same API structure. Credentials are provided as JSON via stdin, and the result is a token.

Example::

```
echo '{"user":"admin","pass":"secret"}' | php index.php auth/login
```

Subsequent authenticated requests use the token as an environment variable:

```
token=your-token php index.php app/customers/list
```

Alternatively, from the command line the token is not required. You can specify the user directly:

Example::

```
user=admin php index.php app/customers/list
```

This shortcut is only available from the command line, never from HTTP. It is not a security boundary: any system user that can execute the PHP scripts can also read the configuration and the data of the instance, so the access to the command line must be restricted with the file system permissions.

3.9 Token Lifecycle & Session Security

SaltOS4 uses token-based authentication for both HTTP and CLI access. Beyond basic login/logout operations, the system enforces session integrity and security through token validation mechanisms.

3.9.1 Token Binding

Each token is bound to the following client attributes:

- **IP address** (‘remote_addr‘)
- **User agent** (‘user_agent‘)

This ensures that a token cannot be reused from another location or browser, protecting against token theft and reuse attacks.

3.9.2 Expiration and Invalidation

Tokens are subject to expiration based on system configuration (‘auth/tokenshortexpires‘ and ‘auth/tokenlongexpires‘). They are invalidated:

- After a period of inactivity, each ‘auth/check‘ renews the expiration up to the maximum lifetime of the token
- On user logout
- When the same user gets a new token (login, password update or password renew)
- When the user has not an active password, for example because it has expired
- When the user is out of its allowed time range or days
- When an administrator deactivates them

A valid token used from another IP address or user agent is rejected.

Expired or invalid tokens are automatically deactivated by ‘crontab_users()‘, that is executed before each token validation.

3.9.3 CLI Considerations

When executing scripts from the command line:

- Tokens can be obtained via ‘auth/login‘ and passed as ‘token=...‘ in subsequent commands.
- Tokens can be bypassed by specifying ‘user=...‘ directly, this shortcut is not available from HTTP.
- The actions ‘setup‘, ‘cron‘, ‘gc‘, ‘integrity‘ and ‘indexing‘ are only allowed from the command line.
- The file system permissions are the real protection: any system user that can execute the PHP scripts can also read the configuration and the data of the instance.

3.9.4 Additional Protections

- The backend checks token validity on every request via ‘current_token()‘, for both HTTP and CLI access.
- The ‘auth/check‘ action (‘checktoken()‘) validates the token and renews its expiration.
- Sessions are tightly coupled to server-side validation and cannot be spoofed via headers alone.

These security measures collectively ensure that only valid users can perform actions and that token reuse or hijacking is effectively prevented.

4 Frontend (web/)

Frontend is a JavaScript SPA in the ‘web/‘ folder.

- ‘index.html‘: loads Bootstrap, SaltOS scripts
- ‘web/js/‘: client-side modules
- ‘web/lib/‘: JS libraries

JavaScript Modules:

- ‘app.js‘: * Application helper module
- ‘auth.js‘: * Authentication helper module
- ‘backup.js‘: * Backup & Autosave helper module
- ‘bootstrap.js‘: * Bootstrap helper module
- ‘common.js‘: * Common helper module
- ‘core.js‘: * Core helper module
- ‘driver.js‘: * Driver module
- ‘filter.js‘: * Filter module
- ‘form.js‘: * Form helper module
- ‘gettext.js‘: * Gettext helper module
- ‘hash.js‘: * Hash helper module
- ‘object.js‘: * Object helper module
- ‘polyfill.js‘: * Global prototype extensions and polyfills for the SaltOS framework
- ‘proxy.js‘: * Proxy module
- ‘push.js‘: * Push & favicon helper module
- ‘storage.js‘: * Storage helper module
- ‘token.js‘: * Token helper module
- ‘window.js‘: * Window helper module

4.1 Logic (core.js and app.js)

SaltOS4’s frontend architecture is built upon two central modules: ‘core.js‘ and ‘app.js‘. Together, they form the foundation for most runtime behavior, enabling dynamic interaction with the backend and managing the user interface.

4.1.1 core.js Overview

The file ‘core.js‘ contains low-level utility functions and system-wide features. It is responsible for:

- Capturing and reporting JavaScript errors, including sourcemapped stack traces.
- Sending AJAX requests, abstracted through ‘saltos.core.ajax(...)‘.
- Creating and manipulating HTML and DOM nodes (‘saltos.core.html(...)‘, ‘saltos.core.append(...)‘).
- Managing configuration options and global parameters.
- Serving as a base for other frontend modules (like auth, filter, window, etc).

This file acts as the technical backbone for the client-side logic.

4.1.2 app.js Overview

The file ‘app.js‘ implements the high-level application layer. It builds on top of ‘core.js‘ and adds features related to UI interaction and view management:

- Registers and manages application views and layouts.
- Handles modal dialogs, toast notifications, and main layout rendering.
- Processes hash-based navigation (‘#app/xyz/...‘) and dispatches actions.

- Manages event hooks (e.g., ‘saltos.app.login‘, ‘saltos.app.ready‘).
- Integrates layout types (‘type1‘ to ‘type3‘) through ‘saltos.driver‘.

It is also responsible for launching the application when loaded in the browser.

4.1.3 Integration and Responsibilities

- ‘core.js‘ provides shared functionality, DOM helpers, AJAX, and error reporting.
- ‘app.js‘ drives the user interface, manages state, and coordinates navigation.
- Modules like ‘auth.js‘, ‘filter.js‘, ‘form.js‘ and others rely on both core and app for foundational logic.

These two modules form the base platform for all SaltOS4 frontend applications.

4.2 Custom Widgets

Each widget of the web client is a constructor registered in ‘saltos.bootstrap.__field‘ using the name of its type, and ‘saltos.bootstrap.fi only dispatches each field of the layout to the constructor of its type. For this reason, any application can provide its own widgets library: it only needs to register the new constructors in a javascript file loaded using the ‘<javascript>‘ node before the ‘<layout>‘ node, and since then, the new types can be used in the layouts as any other widget of SaltOS.

The framework does not know anything about the types of the widgets, the functions that work with the fields of the form (‘saltos.app.get_data‘, ‘saltos.app.check_required‘, ‘saltos.app.form_disabled‘, ‘saltos.form.data‘ and ‘saltos.driver.reset‘) only use the features that each widget stores in the ‘element.saltos‘ object.

4.2.1 The element.saltos object

The native properties of the DOM elements are maintained in the element, and all features and internals added by SaltOS are stored in ‘element.saltos‘, for example, the instances of the libraries used by some widgets (‘codemirror‘, ‘joditeditor‘, ‘tomselect‘, ‘jspreadsheet‘, ...) or the queue used to wait for them.

The data input fields must store the object returned by ‘saltos.bootstrap.__saltos_helper‘:

```
element.saltos = saltos.bootstrap.__saltos_helper(element, field);
```

This object contains the contract of the data input fields, with the default features that can be replaced by each widget after the call to the helper:

- ‘is_input‘: true, used to identify the data input fields.
- ‘old_value‘: the original value, used by ‘get_data‘ to detect the changes, by default ‘field.value‘, it is updated by ‘saltos.form.data‘ when the sync argument is enabled.
- ‘set_value(value)‘: sets the value in the widget, by default ‘element.value = value‘, used by ‘saltos.form.data‘ and ‘saltos.driver.rese
- ‘set_disabled(bool)‘: enables or disables the widget, by default sets or removes the disabled attribute, used by ‘saltos.app.form_d
- ‘get_data()‘: returns the current and the original values as ‘val,‘, normalized to allow to ‘saltos.app.get_data‘ to compare both values in a reliable way, by default ‘element.value,‘.
- ‘check_required()‘: returns an object with ‘value‘ (the value to check), ‘color‘ (the element to colorize), ‘focus‘ (the element to focus) and optionally ‘extra‘ (a function that receives the value, used to colorize other elements), used by ‘saltos.app.check_required‘, by default the element value and the element itself.

4.2.2 Rules

- The data input fields use the helper once, and then they replace the features that need a different behavior.
- The widgets that are not data input fields (buttons, lists, tables, ...) do not use the helper, they only create the object using ‘element.saltos = {}‘ and add their own features, for example, the lists and tables provide ‘set_value‘ and ‘add_value‘.

- The widgets constructed using other widgets do not use the helper again, because the element already contains the object and it would be replaced, losing the features programmed by the original widget, instead of this, they wrap the original features.
- The features use the original value stored in 'old_value' and never the 'field.value', because some widgets receive a copy of the field and 'saltos.form.data' only updates the original one.

4.2.3 Example

The tester application contains an example of a widgets library in the file 'apps/tester/js/rating.js', that provides the rating widget, a data input field that shows five stars and stores the selected value in a hidden input, it replaces all features of the contract:

```
saltos.bootstrap.__field.rating = field => {
    saltos.core.check_params(field, ['id', 'class', 'value', 'disabled', 'required',
                                     'tooltip', 'label', 'color']);
    let color = 'primary';
    if (field.color !== '') {
        color = field.color;
    }
    const obj = saltos.core.html(`
        <div>
            <input type="hidden" id="${field.id}" value="${field.value}" />
            <div>
                <div class="rating d-inline-flex gap-2 px-2 rounded ${field.class}"
                    data-bs-title="${field.tooltip}"></div>
            </div>
        </div>
    `);
    const input = obj.querySelector('input');
    const stars = obj.querySelector('.rating');
    // The hidden input is the data input field, and it receives the default features
    input.saltos = saltos.bootstrap.__saltos_helper(input, field);
    for (let i = 1; i <= 5; i++) {
        const star = saltos.core.html(`
            <button type="button" class="btn btn-link p-0 fs-4 text-${color} bi-star"
                aria-label="${i}"></button>
        `);
        star.addEventListener('click', () => {
            input.saltos.set_value(i);
        });
        stars.append(star);
    }
    // Program the set_value feature, that stores the value and paints the stars
    input.saltos.set_value = value => {
        value = parseInt(value, 10);
        if (!saltos.core.is_number(value)) {
            value = 0;
        }
        input.value = value ? value : '';
        stars.querySelectorAll('button').forEach((star, index) => {
            star.classList.toggle('bi-star-fill', index < value);
            star.classList.toggle('bi-star', index >= value);
        });
    };
    // Program the set_disabled feature, that disables the buttons of the stars
    input.saltos.set_disabled = bool => {
        stars.querySelectorAll('button').forEach(star => {
            star.disabled = bool;
        });
    };
    // Program the get_data feature, the void value and 0 are the same value
    input.saltos.get_data = () => {
```

```
        let val = parseInt(input.value, 10);
        let old = parseInt(input.saltos.old_value, 10);
        if (!saltos.core.is_number(val)) {
            val = 0;
        }
        if (!saltos.core.is_number(old)) {
            old = 0;
        }
        return [val, old];
    };
    // Program the check_required feature, the hidden input can not be colorized or
       focused
    input.saltos.check_required = () => ({
        value: input.value,
        color: stars,
        focus: stars.querySelector('button'),
    });
    if (field.tooltip !== '') {
        saltos.bootstrap.__tooltip_helper(stars);
    }
    // Initialize the widget using the previous features
    input.saltos.set_value(field.value);
    if (saltos.core.eval_bool(field.disabled)) {
        input.saltos.set_disabled(true);
    }
    obj.prepend(saltos.bootstrap.__label_helper(field));
    return obj;
};
```

The rating widget is used in the layout of the tester screen as any other widget, the javascript file is loaded before the layout to register the new type before the creation of the fields:

```
<main>
    <javascript>
        <file eval="true">file_with_hash(file_with_min('apps/tester/js/tester.js'))</file>
        <file eval="true">file_with_hash(file_with_min('apps/tester/js/rating.js'))</file>
    </javascript>
    <layout append="one">
        <row auto="true">
            <rating id="campo47" label="Campo 47 (rating)" required="true"/>
            <rating id="campo47a" label="Campo 47 (rating with value and color)" value="3"
                color="danger"/>
        </row>
    </layout>
</main>
```

And the functions of the framework use the rating widget without any change:

- 'saltos.app.get_data' returns the value as an integer, and detects the changes comparing it with the original value, where the void value and 0 are the same value.
- 'saltos.app.check_required' colorizes the stars container and focuses the first star when the rating is void.
- 'saltos.form.data' and 'saltos.driver.reset' call the 'set_value' feature to paint the stars.
- 'saltos.app.form_disabled' calls the 'set_disabled' feature to disable the buttons of the stars.

The widgets constructed using other widgets can be seen in the core widgets, for example, the tags widget is constructed using the text widget, the onetag widget is constructed using the select widget, and the codemirror and joditeditor widgets are constructed using the textarea widget, in all these cases, the helper is not used again and the features are replaced or wrapped.

4.3 Deployment

Before deploying to production, make sure frontend assets are generated using:

```
make web
```

Production environments should not rely on development mode builds.

To use SaltOS4 from the browser:

- Publish only ‘web/‘, the ‘api/‘, ‘apps/‘ and ‘data/‘ directories must stay outside of the document root
- Inside ‘web/‘:
  - ‘api/index.php‘ → the only entry to the backend, it jumps to the real ‘api/index.php‘
  - ‘apps/‘ → links to the public files of the apps (js, css and pdf), generated by ‘make web‘
- Inside ‘api/‘, symbolic links to:
  - ‘data/‘ → file storage
  - ‘apps/‘ → app definitions
- Configure the web server with the recipe of ‘scripts/‘ that matches it:
  - ‘server.nginx.conf‘ → nginx with PHP-FPM
  - ‘server.apache.conf‘ → Apache with access to the global configuration
  - ‘server.htaccess‘ → Apache on shared hostings where all the code is published
- Validate the result with ‘php api/index.php setup/server <api url>‘, a ‘count‘ of 0 means that nothing private is exposed

As the private directories are not published, the web server does not need access rules. The ‘.htaccess‘ files of ‘api/‘, ‘apps/‘ and ‘data/‘ only deny everything: they are a minimum security for Apache when the full code is published by mistake, and nginx ignores them.

4.4 Offline Support (Service Worker)

SaltOS4 includes a service worker defined in ‘js/proxy.js‘ that transparently enables offline support and network optimization. It plays a key role in improving performance and user experience, even under poor connectivity conditions.

4.4.1 Main Features

- Intercepts all ‘fetch‘ requests and serves them from cache when offline.
- Queues write operations (e.g., POST) and synchronizes them when the network is restored.
- Integrates seamlessly with the core AJAX layer (‘core.js‘), requiring no changes in application logic.
- Logs operations (network, cache, queue, error) using visual debug tools.
- Handles smart caching and fallback strategies depending on the request type.

4.4.2 Internal Structure

The service worker (‘js/proxy.js‘) contains the following key components:

- **‘console_log(message)‘**:
  - Broadcasts messages to all connected clients using ‘postMessage‘, since ‘console.log()‘ is often suppressed in service workers.
- **‘debug(action, url, type, duration, size)‘**:

    - Builds color-coded debug output showing the nature of the request (‘network‘, ‘cache‘, ‘queue‘, ‘error‘), its duration, and response size.

- **‘fetch‘ event listener**:
    - Intercepts all network requests.
    - Determines behavior based on request method and headers.
    - Responds from cache, fetches from the network, or queues operations depending on availability and context.

- **‘message‘ event listener**:
    - Handles external control commands sent by the application. Supported messages include:
        - ‘resetcache‘: clears the entire cache storage.
        - ‘resetqueue‘: clears the queue of pending offline operations.
        - ‘stop‘: unregisters the service worker.
        - ‘hello‘: returns a "hello" reply for testing communication.
        - ‘sync‘: processes all queued requests and tries to send them online.

These capabilities allow external scripts to manage the state and behavior of the proxy dynamically.

4.4.3 Debugging and Observability

To aid debugging:

- All major proxy actions are logged with clear visual indicators.
- Logs are forwarded to client tabs for visibility, even if the browser suppresses SW console output.
- This makes the proxy easier to monitor during development.

4.4.4 Summary

SaltOS4’s service worker acts as a smart, low-level proxy that enables offline functionality and efficient request handling. It ensures smooth interaction with the backend while minimizing the impact of network failures or latency.

4.5 Authentication

SaltOS4 uses JavaScript modules to handle login and authentication on the client side. The core logic is implemented in ‘web/js/auth.js‘, while the login form logic is defined in ‘apps/users/js/login.js‘.

4.5.1 Core Auth Module

The ‘saltos.authenticate‘ module provides these functions:

- ‘authtoken(user, pass)‘: Sends credentials to the backend (‘auth/login‘). If successful, stores the token using ‘saltos.token.set(...)
- ‘checktoken()‘: Validates the current token by calling ‘auth/check‘.
- ‘deauthtoken()‘: Logs out the user and removes the token (‘auth/logout‘).
- ‘authupdate(old, new, renew)‘: Changes the user’s password (‘auth/update‘).

These functions internally use ‘saltos.app.ajax(...)‘, which wraps ‘saltos.core.ajax(...)‘ for simplified usage.

4.5.2 Login Workflow

The ‘saltos.login.authenticate‘ function handles the login form:

- Validates that required fields are filled.
- Retrieves ‘user‘ and ‘pass‘ from the form.
- Calls ‘saltos.authenticate.authtoken(...)‘.
- If the password is expired, shows the form to renew it.
- On failure, shows an error with ‘saltos.app.modal(...)‘.
- On success:
- Redirects to ‘app/dashboard‘ if login was accessed directly.
- Triggers the global event ‘saltos.app.login‘.

This flow enables seamless login via the web UI while leveraging the same token-based backend used for API and CLI access.

4.6 Build System

SaltOS4 provides two build modes for the frontend: production and development. These modes control how JavaScript assets are generated and loaded in the browser.

The build process must be executed from the project root using the Makefile. In production environments, assets should already be generated before deployment.

4.6.1 Overview of Modes

- **Production Mode (‘make web‘)**:
  - Combines and minifies all JavaScript into ‘index.js‘.
  - Combines the service worker proxy and MD5 loader into ‘proxy.js‘.
  - Generates a minified ‘index.html‘ with ‘integrity‘ hashes and MD5-based cache busting.
  - Generates ‘web/apps/‘ with links to the public files of the apps only.
  - Loads minimal assets, optimized for performance and security.
- **Development Mode (‘make devel‘)**:
  - Loads all source files individually from the ‘js/‘ folder.
  - Skips minification, hashing, and integrity checks.
  - Generates an ‘index.html‘ with expanded script references for direct debugging.
  - ‘proxy.js‘ becomes a lightweight loader with ‘importScripts(...)‘.
  - ‘web/apps‘ becomes a link to the full ‘apps/‘ directory, not intended to be published.

4.6.2 Build Process with Makefile

- ‘make web‘:
  - Uses scripts such as ‘fixpath.php‘, ‘md5sum.php‘, ‘uglifyjs‘, ‘sha384.php‘.
  - Populates ‘index.html‘ based on the ‘web/html/index.html‘ template, inserting hashes and integrity attributes.
  - Uses ‘makeapps.php‘ to create ‘web/apps/‘ with one link for each js, mjs, js.map, css and pdf file of the apps (a js file is skipped when its min.js exists, and the composer ‘vendor‘ directories are skipped too) and one link for each file of the ‘files‘ directory of the tester app.
- ‘make devel‘:
  - Uses ‘debug.php‘ to create a loader that references unminified files directly.
  - Generates a simplified ‘proxy.js‘ pointing to the original sources.

Both modes are based on the same HTML loader template found in ‘web/html/index.html‘, which is dynamically populated depending on the mode.

4.6.3 Web Directory Overview

- ‘api/‘: directory with the ‘index.php‘ that jumps to the backend API (‘code/api‘)
- ‘apps/‘: links to the public files of the apps (production) or symlink to ‘code/apps‘ (development)
- ‘html/‘: loader templates for generating ‘index.html‘ and ‘ping.html‘
- ‘img/‘: interface images and logos
- ‘index.html‘: main entry point, minified in production, expanded in development
- ‘index.js‘: combined JavaScript (production only)
- ‘index.js.map‘: sourcemap for ‘index.js‘ (production only)
- ‘proxy.js‘: combined service worker or loader depending on mode
- ‘proxy.js.map‘: sourcemap (production only)
- ‘js/‘: source JavaScript code
- ‘lib/‘: third-party libraries

4.6.4 Comparison Table

Feature	Production Mode (‘make web‘)	Development Mode (‘make devel‘)
Main JS	Combined in ‘index.js‘	Loaded as individual source files
Proxy	Minified ‘proxy.js‘	Lightweight ‘importScripts(...)‘ loader
HTML Loader	Minified ‘index.html‘ with ‘integrity‘	Expanded ‘index.html‘ with raw script tags
Integrity Check	Enabled (SRI hashes)	Disabled (empty ‘integrity‘ attributes)
Cache Busting	Enabled (MD5 hashes)	Disabled
Sourcemaps	Included for debugging	Uses original sources directly
Apps directory	Links to the public files only	Symlink to the full ‘apps/‘
Goal	Efficiency and security	Debugging and development flexibility

5 Application System

Applications in SaltOS4 are modular, defined in folders under ‘apps/‘.

Each folder may contain:

- ‘xml/manifest.yaml‘: declares the apps
- ‘xml/*.xml‘ or ‘xml/*.yaml‘: application definitions
- ‘php/‘, ‘js/‘, ‘locale/‘: optional logic and translations
- ‘dbschema.xml‘, ‘dbstatic.xml‘: database structure and static content

5.1 Manifest Files

The ‘manifest.yaml‘ file defines the metadata and registration details for each SaltOS4 application. It is a declaration file that the system reads to know how to display, categorize, and enable the app. Its use makes the system modular and extensible.

Each ‘manifest.yaml‘ is usually located at ‘apps/<appname>/xml/‘.

5.1.1 General Structure

The general structure of the file is as follows:

```
groups:
    - code: group_code
      name: Group Name
      description: Description
      color: color
      position: number

apps:
    - id: number
      active: 1|0
      code: app_code
      name: App Name
      description: Description
      group: group_code
      position: number
      color: color
      table: main_table
      field: main_field
      subtables: alias:table(foreign_key),...
      has_index: 1|0
      has_control: 1|0
      has_version: 1|0
      has_files: 1|0
      has_notes: 1|0
      has_log: 1|0
      widgets: widget1,widget2
      perms: "*"
      allow: 1
      deny: 0
      folder: folder_name
```

5.1.2 Groups (<group>)

- **Identity:**
  - ‘code‘: unique identifier.
  - ‘name‘: display name.
  - ‘description‘: visible description.
- **Visual (optional):**
  - ‘color‘: Bootstrap class (primary, secondary, success, info, warning, danger).
  - ‘position‘: descending order number (higher appears first).

Note: All visual attributes are optional; if not defined, they are not applied.

5.1.3 Apps (<app>)

- **Identity:**
  - ‘id‘: unique numeric identifier.
  - ‘active‘: 1 (active), 0 (inactive).
  - ‘code‘: internal code (used in URLs, API, etc.).
  - ‘name‘: display name.
  - ‘description‘: short description.

- **Visual (optional):**
  - ‘group‘: group code it belongs to.
  - ‘position‘: order within the group (higher appears first).
  - ‘color‘: Bootstrap class (same as the group’s).
- **Data:**
  - ‘table‘: main associated table.
  - ‘field‘: representative field or computed string.
  - ‘subtables‘: related subtables (alias:table(foreign_key),...)
  - ‘folder‘: folder of the app inside ‘apps/‘, used to resolve the app files.
- **Feature Modules:**
  - ‘has_index‘: enables automatic indexing (creates Mroonga table).
  - ‘has_control‘: enables ownership and sharing control.
  - ‘has_version‘: enables version control (with local blockchain).
  - ‘has_files‘: enables files management.
  - ‘has_notes‘: enables notes management.
  - ‘has_log‘: enables data access logging.
- **Widgets:**
  - ‘widgets‘: list of widgets provided by the app for the dashboard.
- **Permissions:**
  - ‘perms‘: list of permissions (comma-separated or using ‘*‘ as wildcards).
  - ‘allow‘: 1 to force global access.
  - ‘deny‘: 1 to force global denial.

Notes:

All visual attributes are optional; if not defined, they are not applied.

If the permission refers to a permission code without an ‘owner‘ (e.g., ‘main‘, ‘menu‘, ‘create‘), you can simply write the code as is.

For permission codes that support ownership (like ‘list‘, ‘view‘, ‘edit‘, ‘delete‘), you must specify the full code by combining the permission and the owner (e.g., ‘viewall‘, ‘viewuser‘, ‘viewgroup‘), or use a wildcard (e.g., ‘view*‘) to include all ownership levels.

Example:

- ‘main,create,viewall‘ (valid)
- ‘view‘ (invalid → must be ‘viewall‘, ‘viewuser‘ or ‘viewgroup‘, or use ‘view*‘)

Using wildcards (e.g., ‘view*‘) is common to grant all ownership levels.

5.1.4 Permission System

SaltOS4 defines standard permissions in ‘tbl_perms‘:

Code	Owner	Name
main		Main access
menu		Show in menu
create		Create records
widget		Show widget
action		Execute action
config		Configure app
help		Show help
info		Info feature
list	user	List only user records
list	group	List user and group records
list	all	List all records
view	user	View details only user records
view	group	View details user and group records
view	all	View details all records
edit	user	Modify only user records
edit	group	Modify user and group records
edit	all	Modify all records
delete	user	Delete only user records
delete	group	Delete user and group records
delete	all	Delete all records

‘allow: 1‘ forces global access; ‘deny: 1‘ globally denies access.

Permissions codes with an ‘owner‘ require specifying the permission together with the owner. For example, instead of ‘view‘, use ‘viewall‘, ‘viewgroup‘ or ‘viewuser‘.

Wildcards like ‘view*‘ are supported to include all ownership levels at once.

Simple permission codes (those without owner) like ‘main‘, ‘menu‘, ‘create‘ can be used directly.

5.1.5 Feature Modules

The feature modules (‘has_*‘) add extra functionality:

- ‘has_index‘: automatically creates a full-text index table using Mroonga as a storage engine inside MariaDB, enabling high-performance searches without external services like Elasticsearch. This index includes not only the fields of the app’s main table but also any defined subtables, as well as related files and notes for each record. The system keeps the index automatically updated on each data change, ensuring consistent and up-to-date search results across all linked information.
- ‘has_control‘: enables the ownership and sharing control module, adding metadata to each record to store the creator’s ‘user_id‘, ‘group_id‘, and optional lists of additional users and groups with whom the record is shared. It also tracks a creation timestamp. This module allows precise control over who owns and who can access each record at the user and group level. It automatically adds user interface elements to view and manage these permissions directly from the record screens.
- ‘has_version‘: enables automatic version control for the app. Every change to a record is stored as a delta in a dedicated version table, preserving the full history of modifications over time. This version data is secured using a local blockchain to ensure data integrity and prevent tampering. The user interface automatically includes a button on each record’s detail view to access and review the version history, showing who made each change, what was changed, and when.
- ‘has_log‘: activates access and query logging for the app, creating a log table that records every read, write, update, or delete operation performed on the app’s data. This provides complete traceability of data access for auditing purposes. Like ‘has_version‘, a button is automatically added to the record detail view to allow users to consult the access log for each record, showing who accessed the data, from where, and when.
- ‘has_files‘: automatically adds the SaltOS4 file module to the app. This creates an additional table to store files related to each record in the app’s main table. It also automatically updates the user interface to include:
  - a button to upload new files on create and edit screens,
  - a table showing attached files on view and edit screens. This functionality is integrated into the core system so that any app enabling ‘has_files‘ gains built-in file upload, management, and display features without needing extra development.

- ‘has_notes‘: similar to ‘has_files‘, but for notes. It creates a notes table linked to the main table and automatically adds UI elements to allow adding new notes and displaying all notes associated with each record on relevant screens. Like ‘has_files‘, this module is pre-integrated into the core and ready for any app that enables it.

5.1.6 Special Cases

- Apps without ‘group‘: e.g., ‘login‘, ‘dashboard‘ (embedded or functional apps).
- Apps without ‘table‘ and ‘field‘: e.g., ‘login‘, ‘dashboard‘, ‘certs‘ (use backend functions).
- Groups defined in different files are globally merged.
- If visual attributes are not defined → no effect (equivalent to null).

5.1.7 Example

```
groups:
    - code: crm
      name: CRM
      description: Customer Management
      color: success
      position: 4
    - code: sales
      name: Sales
      description: Sales Management
      color: info
      position: 3

apps:
    - id: 1
      active: 1
      code: login
      name: Login
      description: Login screen
      perms: main
      allow: 1
    - id: 2
      active: 1
      code: dashboard
      name: Dashboard
      description: Main dashboard
      perms: main,config,help
      allow: 1
    - id: 50
      active: 1
      code: customers
      name: Customers
      description: Manage customers
      group: crm
      position: 1
      color: success
      table: app_customers
      field: name
      has_index: 1
      has_control: 1
      has_version: 1
      has_files: 1
      has_notes: 1
      has_log: 1
      widgets: last_customers,new_customers
      perms: "*"
    - id: 53
```

```
    active: 1
    code: invoices
    name: Invoices
    description: Manage invoices
    group: sales
    position: 1
    color: info
    table: app_invoices
    field: IF(is_closed, invoice_code, proforma_code)
    subtables: lines:app_invoices_lines(invoice_id),taxes:app_invoices_taxes(invoice_id)
    has_index: 1
    has_control: 1
    has_version: 1
    has_files: 1
    has_notes: 1
    has_log: 1
    widgets: last_7_invoices,invoice_total_by_day
    perms: main,create,editall,viewall,deleteall
```

5.1.8 Usage Notes

- Groups and apps are ordered by ‘position‘ descending; within the same position, alphabetically.
- Visual attributes are optional; if not defined they have no effect and only affect the UI appearance, not backend logic.
- ‘perms‘ supports ‘*‘ and wildcards (e.g., ‘view*‘), the ‘perms‘ attribute controls what actions are available (e.g., ‘main, create, viewall, editall, deleteall‘) or can be set to ‘*‘ for full access.
- The system reads all ‘manifest.yaml‘ files when refreshing the app registry.
- Disabling an app (‘active: 0‘) will hide it from UI but preserve its data in the database.
- Subtables help link detailed records like line items or attachments and are used for indexing relationships.
- Apps without ‘group‘ or without ‘table‘/‘field‘ are valid depending on their function (e.g., embedded apps like ‘login‘ or ‘dashboard‘) and may fetch data directly through backend functions instead of database queries.
- Feature modules activated via ‘has_*‘ automatically create and manage their corresponding tables and features.
- ‘has_index‘ enables full-text indexing using Mroonga, integrated as a storage engine in MariaDB for high-performance searches without external services like Elasticsearch.
- ‘has_version‘ enables automatic version control, storing each change as a delta in a version table, secured with a local blockchain to guarantee data integrity.
- ‘has_control‘ adds ownership metadata (user_id, group_id, sharing) and timestamps for each record.
- ‘has_files‘ and ‘has_notes‘ add UI elements and storage for user-uploaded files and notes.
- ‘has_log‘ creates a log table to register access and queries for the app, providing traceability.
- For permissions requiring ownership, you must specify the full code (e.g., ‘viewall‘ instead of ‘view‘), or use a wildcard (e.g., ‘view*‘).

This manifest system makes SaltOS4 highly modular and easy to extend.

5.2 YAML-based Apps

SaltOS4 allows simple applications to be defined using YAML. This format is ideal for CRUD-style interfaces with basic forms and lists. Below are real-world examples used in the system.

5.2.1 tokenslog.yaml

```
app: tokenslog
```

```
require: apps/common/php/default.php
template: apps/common/xml/default.xml
indent: true
screen: type2
col_class: col-md-6
dropdown: false
list:
    # [id, type, label]
    - [user_id, select, User]
    - [created_at, text, Created]
    - [token, text, Token]
    - [active, boolean, Active]
form:
    # [id, type, label]
    - [active, switch, Active]
    - newline
    - [user_id, select, User]
    - [created_at, datetime, Created]
    - [updated_at, datetime, Updated]
    - [remote_addr, text, Remote Address]
    - [user_agent, text, User Agent]
    - [token, text, Token]
    - [expires_at, datetime, Expires]
select:
    # [id, table, optional field]
    - [user_id, tbl_users]
attr:
    # field:
    #     attr: value
    user_agent:
        col_class: col-md-12
```

5.2.2 configlog.yaml

```
app: configlog
require: apps/common/php/default.php
template: apps/common/xml/default.xml
indent: true
screen: type2
col_class: col-md-6
dropdown: false
list:
    # [id, type, label]
    - [user_id, select, User]
    - [key, text, Key]
form:
    # [id, type, label]
    - [user_id, select, User]
    - [key, text, Key]
    - [val, codemirror, Value]
select:
    # [id, table, optional field]
    - [user_id, tbl_users]
attr:
    # field:
    #     attr: value
    key:
        required: true
        autofocus: true
    val:
        required: true
        mode: json
```

```
        indent: true
        col_class: col-md-12
        height: 5em
```

5.2.3 taxes.yaml

```
app: taxes
require: apps/common/php/default.php
template: apps/common/xml/default.xml
indent: true
screen: type2modal
col_class: col-md-6
dropdown: false
list:
    # [id, type, label]
    - [name, text, Name]
    - [description, text, Description]
    - [value, text, Value]
    - [active, boolean, Active]
    - [default, boolean, Default]
form:
    # [id, type, label]
    - [active, switch, Active]
    - [default, switch, Default]
    - [name, text, Name]
    - [value, float, Value]
    - [description, textarea, Description]
attr:
    # field:
    #     attr: value
    name:
        required: true
        autofocus: true
    description:
        col_class: col-md-12
        height: 5em
```

These examples demonstrate the simplicity of defining apps in YAML.

5.2.4 Advanced YAML file documentation

In SaltOS4, a YAML file defines an application's interface and configuration declaratively. It serves as an intermediate specification that the system transforms into an XML file, which is the format actually processed by SaltOS4.

When a YAML-defined app is loaded, SaltOS4 uses the 'require' and 'template' keys to generate the XML dynamically. This generated XML is stored in the cache directory and includes a comment line like:

```
<!-- source: apps/crm/xml/customers.yaml -->
```

This comment traces the XML back to its original YAML source.

The 'indent' key controls whether indentation is applied when generating the XML output. The 'col_class' key sets a default Bootstrap class for all form fields in the app, unless overridden at the field level via 'attr'.

The 'dropdown' key controls whether action icons are displayed inline or inside a dropdown menu when multiple actions exist for a record.

Lines starting with '#' are comments in YAML and are ignored by the parser.

5.2.5 YAML structure

Below is an example YAML structure:

```
app: myapp
require: apps/common/php/default.php
template: apps/common/xml/default.xml
indent: true
screen: type2modal
col_class: col-md-6
dropdown: false
list:
    - [name, text, Name]
    - [active, boolean, Active]
form:
    - [name, text, Name]
    - [description, textarea, Description]
    - [active, switch, Active]
select:
    # [id, table, optional field]
    - [type_id, app_customers_types]
attr:
    name:
        required: true
        autofocus: true
    notes:
        col_class: col-md-12
        height: 5em
    city:
        col_class: col-md-3
    province:
        col_class: col-md-3
    zip:
        col_class: col-md-3
    country:
        col_class: col-md-3
    code:
        col_class: col-md-3
    email:
        col_class: col-md-4
    phone:
        col_class: col-md-4
    website:
        col_class: col-md-4
    type_id:
        col_class: col-md-3
```

5.2.6 Explanation of YAML fields

- ‘app‘: internal code (unique identifier) for the app
- ‘require‘: PHP file to include as backend logic
- ‘template‘: XML template used as a base for the interface
- ‘indent‘: whether to apply indentation (boolean)
- ‘screen‘: type of layout to use (type1, type2, type3, type1modal, type1fluid, type1full, type2modal)
- ‘col_class‘: default Bootstrap class applied to all form fields
- ‘dropdown‘: whether to force inline icons or use a dropdown menu
- ‘list‘: defines the list columns; each entry is field, type,

- ‘form‘: defines the form fields; each entry is field, type,
- ‘select‘: defines relationships between fields and external tables
- ‘attr‘: specifies additional attributes per field inside ‘form‘

5.2.7 List and Form schema

Both ‘list‘ and ‘form‘ sections use the same schema per field: ‘id, type,‘.

For ‘list‘, supported ‘type‘ values are:

- ‘text‘: displays plain text
- ‘select‘: resolves the field by querying a related table and displaying the linked label
- ‘boolean‘: shows a green or red icon (true if value != 0, false if value == 0)
- ‘hastext‘: like ‘boolean‘, but true if the field contains text, false if empty

For ‘form‘, supported ‘type‘ values are:

- ‘text‘, ‘date‘, ‘time‘, ‘datetime‘, ‘checkbox‘, ‘switch‘, ‘integer‘, ‘float‘, ‘color‘, ‘textarea‘, ‘joditeditor‘, ‘codemirror‘, ‘select‘, ‘multiselect‘, ‘newline‘, ‘hrline‘

All these types correspond to widgets provided by SaltOS4, **except ‘newline‘ and ‘hrline‘**, which are special cases. The ‘newline‘ type injects a ‘ ‘ with ‘col_class="col-12"‘ in the XML, effectively creating a line break in the form layout for grouping fields visually, and the ‘hrline‘ type does the same with an ‘<hr>‘, that adds a visible horizontal line.

5.2.8 Select section

The ‘select‘ section allows mapping form fields to other tables. Each entry is ‘id, table,‘. If ‘optional_field‘ is omitted, SaltOS4 tries to auto-resolve the master field from the target app’s manifest.

5.2.9 Attr section

The ‘attr‘ section adds custom attributes per form field in the XML widget, such as ‘required="true"‘, ‘autofocus="true"‘, ‘height="5em"‘, or overrides like ‘col_class‘.

5.2.10 Summary

The YAML file defines:

- Declarative structure for the app’s form and list
- Default and override attributes for fields
- Table relationships via ‘select‘
- UI behaviors like dropdowns and line breaks

SaltOS4 transforms this YAML into a complete XML interface definition, enabling fast and flexible app development.

5.3 XML-based Complex Apps

SaltOS4 also allows full custom apps defined in XML, often used for more complex business logic and interface customization.

5.3.1 invoices.xml

The following is a small example illustrating the structure of an application XML file:

```
<main>
  <screen>type2modal</screen>
  <title>Invoices</title>
  <navbar>
    .
    .
    .
  </navbar>
  .
  .
  .
</main>
<list>
  .
  .
  .
</list>
<form>
  .
  .
  .
</form>
.
.
.
```

This file contains the following important nodes:

- '<main>': Defines a call to app/customers/main that returns the screen type, literals, and the navbar specification
- '<list default="true">': Defines a call to app/customers/list that returns a cache load from app/customers/list/cache
- '<list id="cache">': Defines a call to app/customers/list/cache that returns the interface of a list screen
- '<list id="data">': Defines a call to app/customers/list/data that returns the data of a list
- '<_form>': This defines a form, there is no way to access it externally, it is for internal use
- '<_data require="php/lib/log.php" eval="true">': This defines a data block of a detail view, not accessible externally, intended for internal use
- '<create>': Defines a call to app/customers/create that returns a cache load from app/customers/create/cache
- '<create id="cache">': Defines a call to app/customers/create/cache that returns the interface of a creation screen
- '<create id="insert">': Defines a call to app/customers/insert that allows inserting a record and returns the status
- '<view>': Defines a call to app/customers/view that returns a cache load from app/customers/view/cache
- '<view id="cache">': Defines a call to app/customers/view/cache that returns the interface of a creation screen
- '<edit>': Defines a call to app/customers/edit that returns a cache load from app/customers/edit/cache
- '<edit id="cache">': Defines a call to app/customers/edit/cache that returns the interface of a creation screen
- '<edit id="update">': Defines a call to app/customers/update that allows updating a record and returns the status
- '<delete>': Defines a call to app/customers/delete that allows deleting a record and returns the status
- '<action id="xxx">': Defines a call to app/customers/xxx that allows to execute the xxx action for this application

5.4 Available Layouts

The interface engine (driver.js) in SaltOS4 supports multiple layout types depending on the application's needs. Layout types are named following this pattern:

type[N][variant]

Where:

- N defines the number of main zones (1, 2, or 3).
- Variants (modal, fluid, full) specify additional layout behavior.

5.4.1 Type1 based layouts

- type1: Simple layout with a single zone (#one), using Bootstrap's fixed-width container.
- type1fluid: Same as type1 but using a full-width Bootstrap container-fluid.
- type1modal: Extends type1 by adding #two as a modal to show detail views, still using fixed-width container.
- type1full: Extends type1fluid by adding #two as a modal for detail views while keeping the full-width container-fluid.

5.4.2 Type2 based layouts

- type2: Two zones layout - #one for the list and #two for details and attachments. Always uses container-fluid.
- type2modal: Extends type2 by adding #three as a modal for attachments.

5.4.3 Type3 layout

- type3: Three zones layout - #one, #two, and #three - allowing all views at once. Always uses container-fluid.

5.4.4 Mobile adaptation

If the window.innerWidth is less than 1200 pixels, the system automatically switches to type1, regardless of the layout configured by the app. This improves usability on mobile devices by simplifying the workflow and avoiding cluttered interfaces.

5.5 Data Model using dbschema.xml

Each application in SaltOS4 can define its own database schema through a 'dbschema.xml' file located in the 'xml/' folder. This file describes the required tables, fields, types, indexes, and relationships.

There is also a central file ('api/xml/dbschema.xml') that defines the **core system tables** used by SaltOS4, including:

- tbl_apps, tbl_perms, tbl_users_apps_perms, tbl_groups_apps_perms: permissions per user and group.
- tbl_uploads: temporary uploaded files.
- tbl_cron: log of scheduled tasks.
- tbl_push: push notifications.
- tbl_trash: deleted records.

5.5.1 Structure of dbschema.xml

A typical 'dbschema.xml' file includes:

- '<table>': Defines a database table.
  - 'name': Table name.
- '<fields>': Contains '<field>' entries:
  - 'name': Column name.
  - 'type': SQL type (supports MySQL and SQLite variants).

- ‘pkey‘: Marks the primary key if ‘true‘.
- ‘fkey‘: Defines a foreign key reference.

- ‘<indexes>‘ (optional): Contains ‘<index>‘ entries listing:
  - ‘fields‘: Comma-separated list of fields in the index.

5.5.2 Example

Example (trimmed here for brevity, see full XML files in repository):

```
<table name="app_taxes">
    <fields>
        <field name="id" type="INTEGER" pkey="true"/>
        <field name="name" type="VARCHAR(255)"/>
        <field name="description" type="TEXT"/>
        <field name="value" type="FLOAT"/>
        <field name="active" type="INTEGER"/>
        <field name="default" type="INTEGER"/>
    </fields>
</table>
```

A taxes table might include:

- Primary key id (integer).
- Fields like name (varchar), description (text), value (float), active (integer), and default (integer).
- No special indexes beyond the primary key (unless defined explicitly).

5.5.3 Automatic Schema Management

SaltOS4 automatically reads the ‘dbschema.xml‘ definitions and compares them against the live database. If it detects changes (new fields, indexes, tables), it applies the required SQL commands **automatically** to synchronize the schema.

This means:

- Developers do **not** need to write migration scripts.
- Admins do **not** need external tools (like phpMyAdmin).
- Schema updates happen transparently during deployment or app installation.

5.5.4 System-Wide and App-Specific Schemas

- The **system-wide schema** (‘api/xml/dbschema.xml‘) defines shared tables.
- Each **app-specific schema** (‘apps/<app>/xml/dbschema.xml‘) defines tables used only by that app.

The system merges both to build the complete data model.

5.5.5 Benefits of dbschema.xml

- Guarantees consistency across MySQL and SQLite setups.
- Acts as the single source of truth for the data structure.
- Enables automatic upgrades without manual SQL.
- Simplifies building custom apps with tailored tables.
- Prevents schema drift between environments.

5.5.6 Complement: dbstatic.xml

SaltOS4 also uses a ‘dbstatic.xml‘ file to define **static data** (preloaded values), like the default permissions. The apps, their groups and their permissions are registered with the same mechanism from the ‘manifest.yaml‘ files, and the setup creates the admin user and the admin group.

Together, ‘dbschema.xml‘, ‘dbstatic.xml‘ and the manifests allow SaltOS4 to fully bootstrap a new installation, ensuring it is immediately usable after setup.

5.6 Static Data using dbstatic.xml

The file ‘dbstatic.xml‘ defines static records that are inserted into the database during system initialization, and synchronized again each time that the file changes. These values are essential for SaltOS4 to operate correctly.

The core file ‘api/xml/dbstatic.xml‘ defines only the table ‘tbl_perms‘, with the available permissions like ‘main‘, ‘create‘, ‘widget‘, ‘action‘, etc., and their optional ownership levels (‘user‘, ‘group‘, ‘all‘).

The rest of the initial data does not come from this file:

- The registered applications, their groups and the permission-to-application assignments come from the ‘manifest.yaml‘ files, that are loaded with the same mechanism.
- The admin user, the admin group and their permission grants are created by the setup.

All this data is loaded by the ‘make setuponly‘ and ‘make setupall‘ targets.

Each ‘<row>‘ inside ‘<table>‘ specifies a record to insert, using attributes like ‘id‘, ‘code‘, ‘name‘, ‘owner‘, and ‘active‘.

This static data ensures consistent behavior across installations and is critical to bootstrapping the system correctly.

5.7 How to Add a New App

Adding a new app in SaltOS4 involves several steps that combine both backend and frontend configuration.

5.7.1 Create the App Folder

Inside code/apps/, create a new folder with the app name, e.g. myapp/. This folder will contain:

- php/: backend PHP logic (optional).
- js/: frontend JavaScript logic (optional).
- xml/: configuration files like manifest.yaml and optionally dbschema.xml, dbstatic.xml.
- locale/: translations (optional).

5.7.2 Define the Manifest

Create xml/manifest.yaml inside the app folder. This file declares the app to the system, specifying:

- Unique id (check unused IDs in existing manifests).
- code, name, description.
- Associated group (like crm, sales, etc).
- Main table, field, and optional subtables.
- Flags like has_index, has_control, has_version, etc.
- Required permissions (perms).

Example:

```
apps:
    - id: 100
      active: 1
      group: crm
      code: myapp
      name: My App
      description: My new application
      table: app_myapp
      field: name
      perms: main,menu,create,list*,view*,edit*,delete*
```

5.7.3 Define the Database Schema

Create xml/dbschema.xml to declare the app's database tables. SaltOS4 will read and apply this schema automatically.

Example for a simple table:

```
<root>
    <tables>
        <table name="app_myapp">
            <fields>
                <field name="id" type="INTEGER" pkey="true"/>
                <field name="name" type="VARCHAR(255)"/>
                <field name="description" type="TEXT"/>
                <field name="active" type="INT(11)"/>
            </fields>
        </table>
    </tables>
</root>
```

5.7.4 Define the Static Data (Optional)

Use xml/dbstatic.xml if the app needs initial records (like predefined types or statuses).

5.7.5 Add the Application Definition

For a YAML-based app, create a file like xml/myapp.yaml to define the screens, forms, and lists.

Example:

```
app: myapp
require: apps/common/php/default.php
template: apps/common/xml/default.xml
indent: true
screen: type2modal
col_class: col-md-6
dropdown: false
list:
    - [name, text, Name]
    - [active, boolean, Active]
form:
    - [name, text, Name]
    - [description, textarea, Description]
    - [active, switch, Active]
attr:
    name:
        required: true
        autofocus: true
    description:
        required: true
```

5.7.6 Ensure Symbolic Links

SaltOS4 uses symbolic links to map directories:

- In ‘api/‘, link ‘apps‘ → ‘../code/apps‘ and ‘data‘ → ‘../code/data‘.
- In ‘web/‘, ‘api/index.php‘ jumps to ‘../code/api/index.php‘ and ‘apps/‘ contains the links generated by ‘make web‘ or ‘make devel‘.

This allows the frontend and backend to access shared app resources consistently.

Summary:

- The backend (api/) accesses apps/ and data/ via local symlinks.
- The frontend (web/) accesses the backend through ‘web/api/index.php‘ and the public files of the apps through ‘web/apps/‘.
- Only ‘web/‘ is published, the rest of the code is not accessible from the browser.

5.7.7 Run Database Setup

Apply the new schema and static data:

```
make setuponly
```

5.7.8 Assign Permissions

Grant access to the new app for users or groups using the admin interface or database configuration.

5.7.9 Test the App

Navigate to the app from the SaltOS4 interface and check that:

- Lists and forms load correctly.
- Data can be inserted, updated, and deleted.
- Permissions are enforced.

This structured process ensures the new app integrates smoothly with the SaltOS4 framework and follows its conventions.

6 Makefile Overview

This ‘Makefile‘ automates building, testing, documentation, and setup tasks in SaltOS4.

6.1 Build Targets

SaltOS4 includes several ‘make‘ targets to manage the build process for both production and development environments. These targets automate the generation of optimized assets, cleanup of temporary files, and preparation of debug-friendly output.

- ‘make web‘: Builds and minifies production assets:
  - Combines and compresses CSS/JS
  - Uses ‘fixpath.php‘, ‘md5sum.php‘, ‘uglifyjs‘, ‘minify‘, ‘sha384.php‘
  - Handles app-specific JS from ‘apps/*/js/*.js‘
  - Generates the ‘proxy.js‘ script with source maps

    - Generates ‘web/apps/‘ with ‘makeapps.php‘
- ‘make devel‘: Prepares a development environment with unminified assets using ‘debug.php‘, and links ‘web/apps‘ to the full ‘apps/‘ directory.
- ‘make clean‘: Deletes generated files:
    - Minified JS, CSS, maps, HTML, ‘proxy.js‘, per-app compiled assets and ‘web/apps‘

6.2 Documentation

The ‘make docs‘ target is used to automatically generate documentation for multiple source areas. It leverages PHP scripts to extract comments and convert ‘.t2t‘ files into PDF. HTML output is also implemented (‘makehtml.php‘) but currently disabled - the calls are commented out in the Makefile for every section.

You can control which documentation files are processed using the ‘file‘ parameter. If no value is passed, all sections are generated by default.

Examples:

- ‘make docs‘ → generates everything
- ‘make docs file=api‘ → generates only API documentation
- ‘make docs file=api,web‘ → generates both API and web documentation

Supported sections:

- ‘api‘: generates the backend doc ‘docs/api.t2t‘ from ‘code/api/php‘
- ‘web‘: generates the frontend doc ‘docs/web.t2t‘ from ‘code/web/js‘
- ‘apps‘: generates the applications doc ‘docs/apps.t2t‘ from ‘code/apps/*/php‘ and ‘code/apps/*/js‘
- ‘utest‘: generates the php unit test doc ‘docs/utest.t2t‘ from ‘utest/‘
- ‘ujest‘: generates the javascript unit test ‘docs/ujest.t2t‘ from ‘ujest/‘
- ‘devel‘: updates the developer manual ‘docs/devel.t2t‘ (version/date only - the rest of this file is written by hand) and regenerates its PDF
- ‘locale‘: regenerates the localization files via ‘scripts/makelocale.php‘
- ‘user‘: regenerates the per-language user manuals (‘docs/user_*.t2t‘) via ‘scripts/makeuser.php‘

This logic is implemented using pattern matching via ‘findstring‘ in the Makefile, allowing comma-separated values to select multiple targets at once.

6.3 Testing

SaltOS4 integrates multiple testing layers to maintain code quality across both PHP and JavaScript components.

Testing is divided into static analysis and unit testing.

- ‘make test‘: Runs static analysis only (no unit tests).
    - PHP: ‘php -l‘, ‘phpcs‘, ‘phpstan‘
    - JavaScript: ‘jscs‘, ‘node -c‘ Accepts variable ‘file=...‘, ‘file=all‘, or none (defaults to modified/untracked git files).
- ‘make utest‘: Executes PHPUnit test suites located in the ‘utest/‘ directory. Supports optional filtering by file.
- ‘make ujest‘: Executes Jest test suites located in the ‘ujest/‘ directory.
    - Cleans snapshot diffs and temporary coverage data
    - Supports full or filtered runs
    - Generates coverage reports
- ‘make ujest_by_parts part=1|2|3‘: Runs the Jest suites in three batches: core modules (‘part=1‘), apps and widgets (‘part=2‘), and screenshots (‘part=3‘).

6.4 Environment Checks

SaltOS4 includes helper targets to verify that the environment is correctly set up. The checks are divided into three levels:

- ‘make checkdirs‘: Validates required symbolic links and directory structure (api/apps, api/data, web/api, web/apps).
- ‘make checkdev‘: Verifies development tools used for building, testing, documentation generation, and asset compilation (node, jest, phpunit, phpcs, uglifyjs, txt2tags, etc.).
- ‘make checkprod‘: Verifies external commands required in a production environment (php, openssl, PDF tools, OCR utilities, certificate tools, etc.).
- ‘make check‘: Executes all checks (‘checkdirs‘, ‘checkdev‘, and ‘checkprod‘).

6.5 Dependency Management

‘make libs‘ executes ‘scripts/checklibs.php‘, which checks for new versions of required PHP and JavaScript libraries, as well as other external software used by the system. This mechanism helps monitor version updates and ensures that all dependencies remain current.

Libraries can be integrated through various methods depending on how they are distributed - for example:

- Via ‘wget‘ from a direct URL
- Via ‘npm‘
- Via Composer for PHP packages

The monitoring is based on the file ‘checklibs.txt‘, where each line defines how to retrieve and compare the current version of a library or tool.

Example line from ‘checklibs.txt‘:

```
phpmailer|https://github.com/PHPMailer/PHPMailer/tags.atom|<title>|
    PHRpdGxlPlBIUE1haWxlciA3LjEuMTwvdGl0bGU+
```

As you can see, this line consists of four fields, using pipes (|) as separators:

- The name of the item (library, command, etc.), used as an identifier
- The URL to fetch the latest version information (can be XML, JSON, or plain HTML)
- A regular expression pattern to match the relevant line that announces the latest version
- A base64-encoded string representing the last matched content, used to detect changes

This tells ‘checklibs.php‘ to:

- Retrieve the GitHub tags feed for PHPMailer in Atom (XML) format
- Match the line containing the version information using the regex
- Compare the base64-encoded result with the previously recorded one to determine if an update is available

This system is useful for tracking updates manually without relying on automatic dependency managers.

6.6 Code Statistics

‘make cloc‘ uses ‘cloc‘ to count lines of code excluding minified and ignored files, you can see an example here:

```
-------------------------------------------------------------------------------
Language                     files          blank        comment           code
-------------------------------------------------------------------------------
PHP                            325           4411          17660          27260
JavaScript                      76           1189           7074          12471
XML                             43            392            677           5514
YAML                            74            151            897           3671
```

```
Python                          27            195             48           1419
Dockerfile                       4            152             60            564
Bourne Shell                     5             18             14            126
HTML                             3              2             13             57
Text                             1              4              0             52
diff                             1              1             35             19
JSON                             1              0              0             13
-------------------------------------------------------------------------------
SUM:                           560           6515          26478          51166
-------------------------------------------------------------------------------
```

6.7 System Setup

These targets manage database configuration, initialization, and full environment resets.

- ‘make setuponly‘: Executes the core system initialization (‘api/index.php setup‘). Applies schema definitions (‘db-schema.xml‘) and static data (‘dbstatic.xml‘). No demo/example data.
- ‘make setupdemo‘: Runs ‘setuponly‘ plus demo and example data (certificates, company data, emails, CRM, HR, purchases, sales).
- ‘make setupmysql‘: Points the database configuration file (‘data/files/config.xml‘) at MariaDB/MySQL. Does not run setup by itself.
- ‘make setupsqlite‘: Points the database configuration file (‘data/files/config.xml‘) at SQLite. Does not run setup by itself.
- ‘make setupunlink‘: Removes the database configuration file (‘config.xml‘).
- ‘make setupclean‘: Cleans data directories and resets the MySQL database (‘DROP‘ + ‘CREATE‘). This operation is destructive.
- ‘make setupall‘: Performs a complete installation cycle:
  - ‘setupclean‘
  - ‘setupmysql‘ + ‘setupdemo‘
  - ‘setupsqlite‘ + ‘setupdemo‘
  - ‘setupunlink‘ Provisions both engines with demo data and leaves no ‘config.xml‘ behind. Intended for development or fresh system provisioning.

6.8 System Actions

These targets trigger internal maintenance operations in SaltOS4. They execute specific backend actions without requiring manual API calls.

- ‘make gc‘: Executes the garbage collection process, cleaning obsolete data, expired sessions, and temporary resources.
- ‘make indexing‘: Rebuilds or updates full-text indexes (including Mroonga-based indexes when enabled).
- ‘make integrity‘: Performs integrity validation checks to ensure data consistency and detect structural issues.
- ‘make cron‘: Executes scheduled tasks defined in ‘cron.xml‘, simulating periodic background execution.

6.9 Local Web Server

These targets run the PHP built-in web server on ‘0.0.0.0:8080‘, serving ‘code/web/‘, in the background. The server uses ‘scripts/router.php‘, which returns a 404 error for the files that do not exist and a 403 error for the directories without index, like apache or nginx do, because ‘php -S‘ returns the index of the parent directory in the first case and a 404 error in the second one.

- ‘make httpstart‘: Starts the server.
- ‘make httpstop‘: Stops the server.
- ‘make httpstatus‘: Shows the PID of the running server, if any.

6.10 Docker

All Docker targets run ‘docker compose‘ from ‘scripts/‘, one profile per group of targets. See ‘CONTRIBUTING.md‘ for details.

Profile	Targets	Purpose
‘devel‘	‘develbuild‘, ‘develstart‘, ‘develstop‘, ‘develogs‘, ‘develstatus‘, ‘develbash‘	SQLite + PHP built-in server, for de
‘server‘	‘serverbuild‘, ‘serverstart‘, ‘serverstop‘, ‘serverlogs‘, ‘serverstatus‘, ‘serverbash‘	nginx + PHP-FPM + MariaDB, produ
‘demos‘	‘demosbuild‘, ‘demosstart‘, ‘demosstop‘, ‘demoslogs‘, ‘demosstatus‘, ‘demosbash‘	Public demo site, one SQLite instar
‘test‘	‘teststart‘, ‘teststop‘, ‘testlogs‘, ‘teststatus‘	MSSQL, PostgreSQL and GreenMai
‘db‘	‘dbbuild‘, ‘dbstart‘, ‘dbstop‘, ‘dblogs‘, ‘dbstatus‘, ‘dbbash‘	MariaDB + Mroonga as a host servi

6.11 System langs

This rule runs the ‘scripts/checklangs.py‘ script to validate the integrity of all translation files across languages and groups.

To run the integrity check, use:

```
make langs
```

This will:

- Analyze all ‘manifest.yaml‘, ‘.xml‘, ‘.yaml‘, and ‘.js‘ files in every app group
- Load all ‘messages.yaml‘ files from ‘api/locale/‘ and ‘apps/<group>/locale/‘
- Detect duplicate translation keys:
  - Within the same file
  - Between the global and any app group
  - Between two groups (only with ‘–strict‘)
- Report any overlapping definitions that may need cleanup

To perform additional per-language analysis manually, you can call the script with options:

- ‘–lang <lang>‘: language to check (e.g. ‘ca_ES‘)
- ‘–filter missing|present|missing_but_in_other_group|unused‘
- ‘–group <group>‘: restrict to a specific app group
- ‘–csv <file.csv>‘: export results to a CSV file
- ‘–strict‘: detect duplicate keys across app groups

Examples:

Validate global integrity across all languages:

```
make langs
```

Run language-specific analysis manually:

```
python3 scripts/checklangs.py --lang ca_ES --group crm --filter missing
```

Strict duplicate detection between groups:

```
python3 scripts/checklangs.py --strict
```

6.12 Script Directory Overview

This section describes the purpose of each file in the ‘scripts/‘ directory, grouped by functionality.

6.12.1 Build and Asset Generation

- ‘debug.php‘: Generates a development-mode frontend loader that references unminified JavaScript sources.
- ‘fixpath.php‘: Adjusts file paths in generated HTML/JS/CSS files to match the deployment structure.
- ‘md5sum.php‘: Generates MD5 checksums for one or more files, typically used for cache-busting mechanisms.
- ‘sha384.php‘: Calculates SHA-384 hashes for files to be used in Subresource Integrity (SRI) attributes in generated HTML.
- ‘maket2t.php‘: Scans a source directory and extracts ‘/** … */‘ documentation comments from each file to generate a ‘.t2t‘ documentation file.
- ‘makehtml.php‘: Converts a ‘.t2t‘ documentation file into an HTML file using ‘txt2tags‘.
- ‘makepdf.php‘: Converts a ‘.t2t‘ documentation file into a ‘.pdf‘ using LaTeX.
- ‘imagest2t.php‘: Processes image resources for inclusion in ‘.t2t‘ documentation outputs.
- ‘updatet2t.php‘: Updates the version and date lines of a ‘.t2t‘ documentation file (used, for example, in ‘devel.t2t‘).
- ‘makelocale.php‘: Generates the per-app help PDFs from ‘apps/<app>/locale/<lang>/*.t2t‘, only for the sources newer than their PDF.
- ‘makeuser.php‘: Assembles the per-language user manuals (‘docs/user_*.t2t‘) by concatenating each app’s hand-written help snippets (‘apps/<app>/locale/<lang>/*.t2t‘), then generates their PDFs.
- ‘make_instance.sh‘: Creates a new runnable SaltOS4 instance by linking the files of ‘code/‘, creating its own ‘web/api/‘ and ‘web/apps/‘ directories with links to the contents of ‘code/web/‘, preparing directories like ‘data/‘ and ‘tmp/‘, and applying correct permissions.
- ‘migrate_instance.sh‘: Migrates an instance created by an old ‘make_instance.sh‘, that publishes the full ‘api/‘ and ‘apps/‘ directories, to the current layout, without touching its data.
- ‘makeapps.php‘: Creates the ‘web/apps/‘ directory with links to the public files of the apps, used by ‘make web‘.
- ‘router.php‘: Router for the PHP built-in web server, returns the same 403 and 404 errors that apache or nginx.
- ‘server.nginx.conf‘, ‘server.apache.conf‘, ‘server.htaccess‘: Web server recipes for nginx with PHP-FPM, Apache and Apache on shared hostings, to be adapted and validated with ‘setup/server‘.

6.12.2 Testing and Quality

- ‘jest.config.js‘: Configuration file for running JavaScript unit tests with Jest.
- ‘jest_coverage.php‘: Processes Jest coverage reports and formats them into a readable summary.
- ‘jest_tester.php‘: Parses the layout structure from ‘apps/tester/xml/tester.xml‘, checks for duplicate widget definitions, and exports the result as ‘/tmp/tester.json‘.
- ‘jscs.json‘: Defines JavaScript coding standards used by the JSCS style checker.
- ‘phpcs.xml‘: Configuration file for PHP_CodeSniffer to enforce PHP coding standards.
- ‘phpstan.neon‘: Configuration file for PHPStan, specifying analysis rules and paths for static code analysis.
- ‘phpunit.xml‘: Configuration file for PHPUnit, defining test directories, bootstrap files, and execution settings.
- ‘checklangs.py‘: Scans XML/YAML/JS files for translation keys and detects missing or duplicated strings across apps and language groups.
- ‘movelangs.py‘: Moves selected translation keys from a group-specific ‘messages.yaml‘ file to the global one without altering file structure.
- ‘jest-image-snapshot-diff-on-update.patch‘: Local patch for the ‘jest-image-snapshot‘ npm package (applied via ‘make

ujest_patch‘) so that ‘-u‘ also writes a diff image instead of silently overwriting the baseline.

6.12.3 Database and Migration

- ‘database_export.sh‘: Exports the system database content to a portable format.
- ‘database_import.sh‘: Imports a previously exported database into the system.
- ‘migrate_v3_to_v4.php‘: Migrates a SaltOS system from version 3 to version 4, adapting both data structures and file organization.
- ‘migrate_mysql_to_sqlite.php‘: Migrates data from a MySQL/MariaDB database to SQLite format.
- ‘migrate_sqlite_to_sqlite.php‘: Performs internal SQLite-to-SQLite migration tasks.

6.12.4 Dependency and Library Management

- ‘checklibs.php‘: Validates the current versions of required libraries by parsing ‘checklibs.txt‘, performing HTTP requests, and comparing base64-encoded version strings. Updates the file if changes are detected.
- ‘checklibs.txt‘: Contains the list of tracked libraries, including their version URLs, regex patterns, and base64-encoded reference values.

6.12.5 Docker and Containerization

- ‘docker-compose.yaml‘: Defines Docker services and profiles for development, server, testing, demos and database environments.
- ‘Dockerfile.db‘: Docker image definition used for the ‘db‘ profile, MariaDB with Mroonga built from source.
- ‘Dockerfile.devel‘: Docker image definition used for the development profile.
- ‘Dockerfile.server‘: Docker image definition used for the production/server profile.
- ‘Dockerfile.demos‘: Docker image definition for the ‘demos‘ profile that runs demos.saltos.org - nginx + PHP-FPM + SQLite, provisioning one isolated instance per visitor IP hash on first request.
- ‘demos_index.php‘: Entry point of the ‘demos‘ image. Hashes the visitor’s IP, and on first visit creates and fully sets up a new instance under that hash in the ‘instances/private/‘ directory, publishes its ‘web/‘ directory with a link in the ‘instances/public/‘ directory (the document root) and redirects to it.
- ‘demos_trash.php‘: Runs hourly via cron inside the ‘demos‘ container, moving instances untouched for more than 7 days into ‘instances/removed/‘, and then synchronizing the public directory.
- ‘demos_sync.php‘: Synchronizes ‘instances/public/‘ with ‘instances/private/‘, creating the links of the new instances and removing the links of the instances that not exist. Used by ‘demos_index.php‘ and ‘demos_trash.php‘, and can be executed from the ‘instances‘ directory too.
- ‘wait_index.html‘: Placeholder "starting up" page served while ‘devel‘ and ‘server‘ images provision an instance on first boot.